

SELECTED TOPICS IN COMPUTER SCIENCE — EMBEDDED SYSTEMS

RadarBot

Ultrasonic Radar System · Project Report

PROJECT TEAM

Seif Mohamed Fouad Makled

Antoni Ashraf Mikhael

Third Year · CS-AI Program · May 2026

web dashboard

The web dashboard is a browser-based interface that replaces the standalone Python logger with a fully visual experience. Built with Flask, it runs locally on the connected PC and is accessed by opening `http://127.0.0.1:5000` in any browser.

Instead of watching raw text scroll through a terminal, the dashboard presents the radar data in real time — displaying the current sweep angle, measured distance, operating mode, and buzzer state as live metrics. It also maintains a scrolling history of the most recent logged readings pulled directly from the SQLite database, so you can see how the data accumulates over time without opening a separate database viewer.

The dashboard operates in two modes. When the Arduino is connected and running, it enters **Live mode** — reading the serial stream, logging one record per second to `radar_logs.db`, and pushing every reading to the browser instantly via Server-Sent Events. If no Arduino is available (wrong port, unplugged, or the port is held by another program), it automatically falls back to **Simulation mode**, generating synthetic data so the interface remains fully demonstrable without hardware. A status indicator in the top corner always makes it clear which mode is active.

The dashboard effectively combines the jobs of the Python logger and the Processing visualization into a single tool — no extra software required beyond a web browser.



01 PROJECT OVERVIEW

RadarBot is a real-time proximity sensing and visualization system built entirely on a single Arduino board. At its core, a micro servo motor sweeps an HC-SR04 ultrasonic sensor across a 150-degree arc, firing sound pulses and measuring how long each echo takes to return. That round-trip time translates directly into distance — updated every 30 milliseconds, at every degree of rotation.

When something enters the 30-centimeter danger zone, a passive buzzer fires a 2 kHz alarm. Simultaneously, the Arduino streams angle and distance data over USB serial to two separate software layers running on a connected PC: a Processing sketch that renders a classic green radar display, and a Python script that logs every reading to a local SQLite database for later analysis.

A joystick module adds a layer of interactivity — one button press hands the operator direct control over the servo angle, and another press gives it back to the automatic sweep. The project brings together sensor interfacing, PWM motor control, real-time serial communication, and multi-layer software integration in a way that is both practically functional and visually compelling.

02 HARDWARE COMPONENTS

The system runs on off-the-shelf components, all wired through a single breadboard that acts as the shared power and ground bus. The table below lists each component and its role in the system.

Component	Role
Arduino (Uno / Mega)	Central microcontroller — runs all firmware
HC-SR04 Ultrasonic Sensor	Measures distance via sound echo timing (Trig: Pin 8, Echo: Pin 9)
SG90 Micro Servo Motor	Rotates the sensor tower across the scanning arc (Signal: Pin 10)
Passive Buzzer	Piezo alarm — 2 kHz tone when object < 30 cm (Pin 7)
Joystick Module	X-axis controls angle in manual mode; button toggles mode (A0, Pin 4)
Breadboard + Cables	Power rail distribution and component interconnection

03 SYSTEM ARCHITECTURE

RadarBot is structured across three distinct software layers, each with a clear responsibility. The Arduino is the only layer that talks directly to hardware — everything above it communicates through the USB serial port.

Arduino Firmware

The on-device layer. It reads the ultrasonic sensor, drives the servo via PWM, monitors the joystick button for mode changes, and activates the buzzer when the distance threshold is crossed. Every 30 ms, it serialises the current angle and distance and writes them to the USB serial port at 9600 baud.

Processing — Radar Display

A desktop sketch that listens on the same serial port and renders a radar visualization in real time: a sweeping green line, concentric range rings at 20 cm intervals out to 100 cm, and a red marker wherever an object is detected. Distance and angle are shown numerically at the bottom of the screen.

Python + SQLite — Data Logger

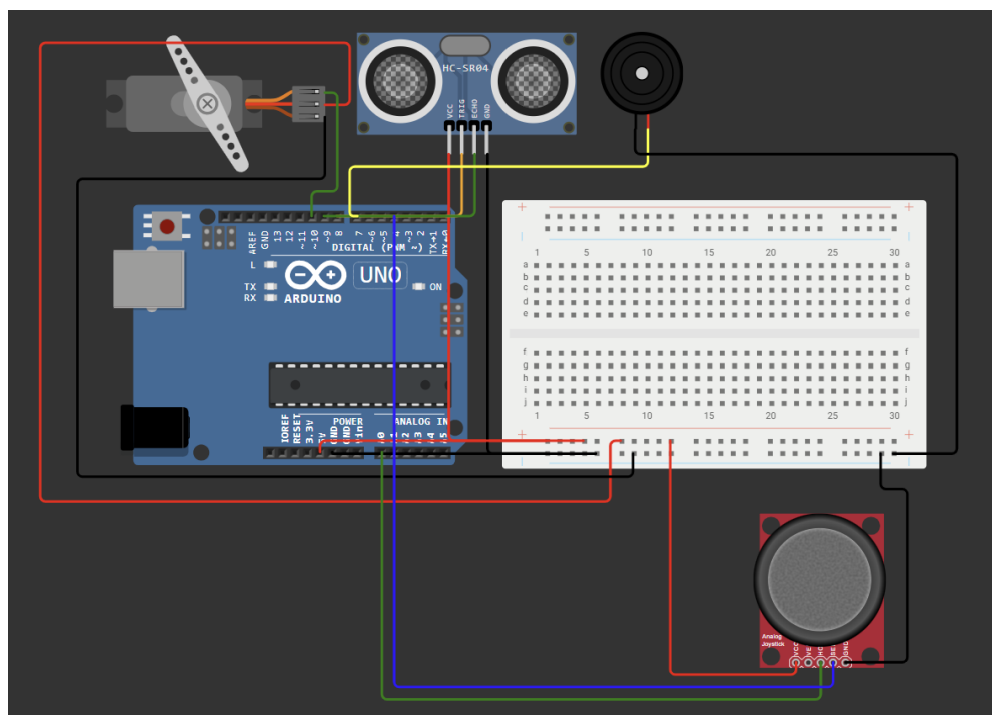
A Python script that reads the serial stream, parses each packet, and inserts a timestamped record into a local SQLite database (`radar_logs.db`) once per second. The database schema captures angle, distance, operating mode, and buzzer state — queryable with the SQL scripts in the SQL Queries folder.

One important constraint: Processing and the Python logger both open the same serial port, so only one can run at a time. This is expected behaviour — the operator closes the visualization when they want to log data, and vice versa.

04 CIRCUIT DIAGRAM

The figure below shows the complete wiring layout as simulated in Wokwi. All component power lines connect to the breadboard's shared 5V rail, which is fed from the Arduino's onboard 5V pin. Ground lines share the breadboard's GND rail, tied back to the Arduino GND. Signal wires run directly between each component and its designated Arduino pin.

Figure 1: Wokwi Circuit Simulation — Full Wiring Layout



05 PROCESSING RADAR UI

The Processing sketch renders a full-screen radar display modelled on classic military-style PPI (Plan Position Indicator) scopes. A bright green sweep line rotates in sync with the physical servo. Five concentric arcs mark the 20, 40, 60, 80, and 100 centimetre range rings. When the sensor detects an object within range, a red line segment appears at the measured distance and holds briefly as the sweep continues past — leaving a trace that fades naturally with each pass.

An out-of-range reading (no echo within 25 ms) renders as an orange marker at the edge of the display rather than leaving a false close-range detection. Angle and distance are printed numerically at the bottom of the screen alongside the team name.

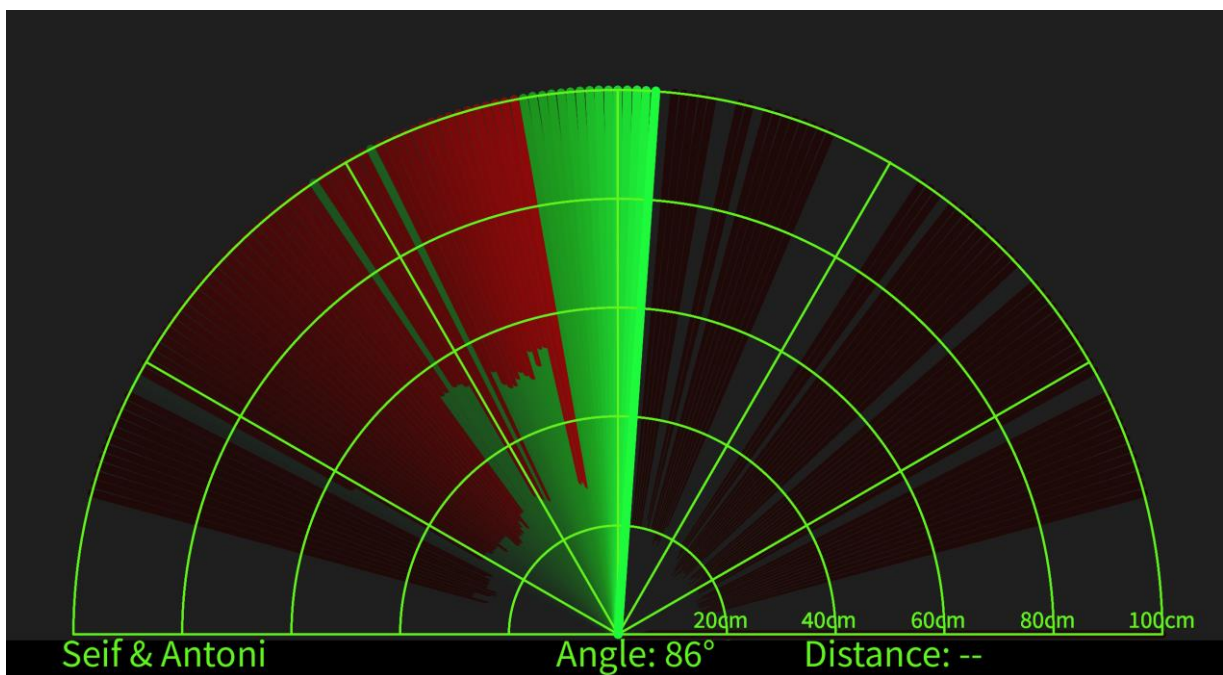


Figure 2: Real-Time Radar Visualization — Processing Sketch

06 DATABASE LOGGING

The Python logger connects to the Arduino's serial port at 9600 baud and reads incoming packets continuously. To avoid flooding the database with redundant data, it applies a one-second rate limit — writing exactly one record per second regardless of how fast the Arduino is transmitting. Each record carries five fields:

Field	Type	Description
timestamp	TEXT	ISO 8601 timestamp (microsecond precision)
angle	INTEGER	Servo angle in degrees (15 – 165)
distance	INTEGER	Measured distance in cm (-1 = no echo / out of range)
mode	INTEGER	0 = Automatic sweep 1 = Manual joystick control
buzzer_active	INTEGER	0 = Buzzer OFF 1 = Buzzer ON

The SQL Queries folder ships five ready-made queries covering the most common analysis tasks: filtering by angle range, distance threshold, operating mode, buzzer state, and time window. DB Browser for SQLite is the recommended tool for browsing the database visually.

DB Browser for SQLite - C:\Users\SeifMaklad\OneDrive\Desktop\Ultrasonic-Radar-System\radar_logs.db

File Edit View Tools Help

New Database Open Database Write Changes Revert Changes Undo Open Project

Database Structure Browse Data Edit Pragas Execute SQL

Table: radar_logs Filter in any column

	id	timestamp	angle	distance	mode	buzzer_active
	Filter	Filter	Filter	Filter	Filter	Filter
1644	1644	2026-05-21T16:20:36.540971	126	-1	0	0
1645	1645	2026-05-21T16:20:37.547928	106	-1	0	0
1646	1646	2026-05-21T16:20:38.571813	86	-1	0	0
1647	1647	2026-05-21T16:20:39.590951	66	-1	0	0
1648	1648	2026-05-21T16:20:40.617366	46	-1	0	0
1649	1649	2026-05-21T16:20:41.676464	25	-1	0	0
1650	1650	2026-05-21T16:20:42.706749	25	-1	0	0
1651	1651	2026-05-21T16:20:43.740073	45	-1	0	0
1652	1652	2026-05-21T16:20:44.752127	69	62	0	0
1653	1653	2026-05-21T16:20:45.754773	89	-1	0	0
1654	1654	2026-05-21T16:20:46.792033	111	176	0	0
1655	1655	2026-05-21T16:20:47.805209	134	-1	0	0
1656	1656	2026-05-21T16:20:48.823155	164	7	0	1
1657	1657	2026-05-21T16:20:49.829862	136	7	0	1
1658	1658	2026-05-21T16:20:50.837887	111	178	0	0
1659	1659	2026-05-21T16:20:51.883247	90	-1	0	0
1660	1660	2026-05-21T16:20:52.895503	66	7	0	1

1,644 - 1,660 of 1,716 Go to: 1

Figure 3: SQLite Database — radar_logs Table (DB Browser for SQLite)

07 OPERATION MODES

Automatic Mode

Default on startup. The servo steps one degree every 30 ms, bouncing between 15° and 165°. The sensor fires at each step and the result is immediately streamed out over serial. No user input required — the radar runs on its own until the joystick button is pressed.

Manual Mode

Activated by pressing the joystick button (Pin 4). The sweep stops and the joystick's X-axis takes over, mapping the analog range (0 – 1023) directly to servo angle (15° – 165°). The sensor still fires continuously at whatever angle the joystick is pointing. Another press of the button returns to automatic.

Buzzer Alarm

Active in both modes. Whenever the measured distance is between 1 and 29 cm, the Arduino calls `tone(7, 2000)` — a 2 kHz pulse on the buzzer pin. The moment the object moves away or disappears, `noTone(7)` cuts it off immediately. A distance of -1 (no echo detected) is treated as out-of-range and does not trigger the alarm.

08 CONCLUSION

RadarBot turned out to be more than the sum of its parts. What started as a handful of cheap sensors and a breadboard became a system that operates across three software layers simultaneously — each one dependent on the others getting their timing and formatting exactly right.

The most demanding part was not the hardware, but the serial protocol: getting the Arduino, Processing, and Python to agree on data format, baud rate, and port access without stepping on each other. That constraint — only one program can own the port at a time — forced a clean architectural decision early on and shaped the rest of the design.

The end result is a working proximity radar: it sweeps, it detects, it alarms, it visualizes, and it logs. Every piece of functionality was built incrementally and tested against real hardware. That is exactly the kind of iterative, constraint-driven engineering that embedded systems demand.